
Université des Sciences et de la Technologie d'Oran MB

Faculté MI - Département d'Informatique -

ING4-DS

GenerativeAI

RAPPORT des TPs

TP1 — DCGAN & WGAN | TP2 — cGAN | TP3 — StyleGAN | TP4 — VAE

BEKRI Yousra Ouissal

Introduction

Ce rapport regroupe l'ensemble des travaux pratiques du module de **Generative AI**. Ces TPs couvrent les principales architectures de modèles génératifs, depuis les réseaux antagonistes génératifs (GANs) jusqu'aux auto-encodeurs variationnels (VAE), en passant par les variantes conditionnelles et stylistiques.

Les travaux pratiques sont organisés de manière progressive :

- **TP1 — DCGAN** : GAN convolutif profond sur des images de briques LEGO.
- **TP1 — WGAN** : GAN avec distance de Wasserstein pour une meilleure stabilité.
- **TP2 — cGAN** : GAN conditionnel pour contrôler la génération par classe (MNIST).
- **TP3 — StyleGAN** : Architecture avancée avec mapping network et injection de style (CIFAR-10).
- **TP4 — VAE** : Auto-encodeur variationnel avec espace latent structuré (MNIST), deux dimensions testées (2D et 20D).

Frameworks	TensorFlow / Keras • PyTorch
Datasets	LEGO Bricks (Kaggle) • MNIST • CIFAR-10

TP1 — DCGAN : Deep Convolutional GAN

1.1 Présentation et Objectifs

Le **DCGAN (Deep Convolutional Generative Adversarial Network)** est une extension des GANs originaux qui utilise des couches de convolution pour améliorer la qualité des images générées. Ce TP implémente un DCGAN sur le dataset **LEGO Bricks** (Kaggle), composé d'images de briques LEGO en niveaux de gris.

1.2 Dataset et Prétraitement

Paramètre	Valeur
Dataset	LEGO Brick Images (Kaggle)
Mode couleur	Niveaux de gris (grayscale)
Taille d'image	64 × 64 pixels
Batch size	32
Normalisation	$[-1, 1]$ via $(x / 127.5) - 1.0$

La normalisation entre $[-1, 1]$ est indispensable car la couche de sortie du générateur utilise une activation `tanh` dont la plage de sortie est exactement $[-1, 1]$. Cela assure la cohérence entre les données d'entraînement et les images générées.

1.3 Architecture du Discriminateur

Le discriminateur reçoit une image 64×64 en entrée et produit une probabilité (0 = fausse, 1 = réelle). Il est construit comme un CNN classique avec des convolutions successives qui réduisent la résolution spatiale tout en augmentant la profondeur.

Couche	Configuration
Conv2D — 1	64 filtres, kernel 4×4, stride 2, padding same — LeakyReLU(0.2) + Dropout(0.3)
Conv2D — 2	128 filtres, kernel 4×4, stride 2 — BatchNorm(0.9) + LeakyReLU(0.2) + Dropout(0.3)
Conv2D — 3	256 filtres, kernel 4×4, stride 2 — BatchNorm(0.9) + LeakyReLU(0.2) + Dropout(0.3)
Conv2D — 4	512 filtres, kernel 4×4, stride 2 — BatchNorm(0.9) + LeakyReLU(0.2) + Dropout(0.3)
Conv2D — 5 (sortie)	1 filtre, kernel 4×4, stride 1, activation sigmoid — Flatten

- **LeakyReLU(0.2)** : Contrairement au ReLU classique, le LeakyReLU laisse passer une fraction (0.2) du signal négatif, ce qui évite le problème des neurones « morts » (dying ReLU) et améliore la propagation des gradients.
- **BatchNormalization(momentum=0.9)** : Normalise les activations de chaque mini-batch, stabilise l'entraînement et accélère la convergence. Le momentum=0.9 contrôle la vitesse de mise à jour de la moyenne mobile.
- **Dropout(0.3)** : Désactive aléatoirement 30% des neurones à chaque forward pass (uniquement en training), ce qui régularise le discriminateur et l'empêche de dominer trop rapidement le générateur.
- **use_bias=False** : Le biais est désactivé car BatchNormalization remplit déjà ce rôle en ajoutant un décalage appris.

1.4 Architecture du Générateur

Le générateur part d'un vecteur de bruit latent de dimension 100 (`latent_dim=100`) et génère une image 64×64 en niveaux de gris via des convolutions transposées (upsampling progressif).

Couche	Configuration
Input	Vecteur z de dimension 100 — Reshape en (1, 1, 100)
Conv2DTranspose — 1	512 filtres, kernel 4×4, stride 1, padding valid — BatchNorm + LeakyReLU(0.2) → (4, 4, 512)
Conv2DTranspose — 2	256 filtres, kernel 4×4, stride 2, padding same — BatchNorm + LeakyReLU(0.2) → (8, 8, 256)
Conv2DTranspose — 3	128 filtres, kernel 4×4, stride 2, padding same — BatchNorm + LeakyReLU(0.2) → (16, 16, 128)
Conv2DTranspose — 4	64 filtres, kernel 4×4, stride 2, padding same — BatchNorm + LeakyReLU(0.2) → (32, 32, 64)
Conv2DTranspose — 5 (sortie)	1 filtre, kernel 4×4, stride 2, activation tanh → (64, 64, 1)

1.5 Fonction de Perte et Optimisation

BinaryCrossentropy — Classification binaire

Le discriminateur effectue une classification binaire :

- Image réelle → label = 1
- Image générée → label = 0

Perte du discriminateur : $d_loss = (d_real_loss + d_fake_loss) / 2$

Perte du générateur : $g_loss = BCE(ones, fake_predictions)$ — tromper le discriminateur

Optimisateur : Adam(lr=0.0002, beta_1=0.5)

→ beta_1=0.5 (au lieu de 0.9 par défaut) réduit les oscillations adversariales

1.6 Boucle d'Entraînement

La boucle utilise deux `tf.GradientTape()` simultanés (un pour le générateur, un pour le

discriminateur). À chaque step :

- **1.** Génération d'un vecteur de bruit latent aléatoire.
- **2.** Génération d'images artificielles par le générateur.
- **3.** Prédiction du discriminateur sur les vraies et fausses images.
- **4.** Calcul des pertes de chaque réseau.
- **5.** Rétropropagation et mise à jour des poids via les optimiseurs.

Un callback `GANMonitor` affiche 9 images générées à la fin de chaque epoch en utilisant un **seed fixe**, ce qui permet de visualiser la progression de l'apprentissage de manière cohérente.

1.7 Paramètres d'Entraînement

Paramètre	Valeur
Epochs	100 (version complète) / 10 (version rapide)
Batch size	32
Latent dimension	100
Learning rate	0.0002
Beta_1 (Adam)	0.5
Activation sortie G	tanh --- sortie dans [-1, 1]
Activation sortie D	Sigmoid --- probabilité [0, 1]

TP1 — WGAN & WGAN-GP : Wasserstein GAN

2.1 Motivations du WGAN

Le GAN classique souffre de plusieurs problèmes majeurs : instabilité d'entraînement, **mode collapse** (le générateur produit toujours les mêmes images), et disparition des gradients quand le discriminateur devient trop fort. Le **WGAN** réforme le jeu adversaire en utilisant la **distance de Wasserstein-1 (Earth Mover's Distance)** comme métrique.

Différences clés DCGAN vs WGAN

DCGAN → WGAN

Discriminateur (classification 0/1) → Critic (score réel, pas de sigmoïde)

Perte BCE → Distance de Wasserstein : $-E[\text{real}] + E[\text{fake}]$

Pas de contrainte → Contrainte 1-Lipschitz requise

1 update D par step → $n_critic=5$ updates D par step G

Sigmoid en sortie → Sortie linéaire (pas d'activation)

2.2 Architecture — Critic (WGAN)

Le critic a la même architecture convolutive que le discriminateur DCGAN, à une différence fondamentale près : **aucune activation sigmoïd en sortie**. Le critic produit un **score réel** (non borné) qui représente la « réalité » de l'image plutôt qu'une probabilité.

2.3 Contrainte de Lipschitz — Weight Clipping

Pour que la distance de Wasserstein soit bien définie, le critic doit être **1-Lipschitz** (sa variation globale doit être bornée). La méthode originale WGAN impose cette contrainte par **weight clipping** : tous les poids du critic sont clippés dans $[-c, +c]$ après chaque mise à jour.

Implémentation du Weight Clipping

`clip_value = 0.01` (valeur recommandée par les auteurs)

Après chaque update du critic :

```
for var in self.critic.trainable_variables:
    var.assign(tf.clip_by_value(var, -clip_value, clip_value))
```

Effet : force la norme des gradients à rester bornée → critère 1-Lipschitz satisfait

2.4 Boucle d'Entraînement WGAN

La boucle d'entraînement entraîne le critic **$n_critic=5$ fois** pour chaque mise à jour du générateur, afin de s'assurer que le critic est suffisamment bien entraîné avant que le générateur ne reçoive son signal de gradient.

Paramètre	Valeur WGAN
n_critic	5 (updates critic par update générateur)
clip_value	0.01
Learning rate	0.0002
Beta_1 (Adam)	0.5
Perte critic	-mean(real_scores) + mean(fake_scores)
Perte générateur	-mean(fake_scores)

2.5 WGAN-GP — Gradient Penalty

Le weight clipping du WGAN original présente des limitations : il peut entraîner un **capacity underuse** (le critic ne peut pas exploiter toute sa capacité) et des **gradients qui s'accumulent aux extrémités $\pm c$** . La variante **WGAN-GP (Gradient Penalty)** remplace le weight clipping par une pénalité douce sur la norme du gradient.

Gradient Penalty — Formule

Pénalité calculée sur des images interpolées entre vraies et fausses :

$\alpha \sim \text{Uniform}(0, 1)$ — coefficient d'interpolation aléatoire
 $\text{interpolates} = \alpha * \text{real} + (1 - \alpha) * \text{fake}$

$\text{gradient} = \nabla(\text{critic}(\text{interpolates}))$
 $\text{norm} = \|\text{gradient}\|_2$
 $\text{GP} = E[(\text{norm} - 1)^2]$ — pénalise l'écart à la norme 1

$\text{c_loss} = -E[\text{real_scores}] + E[\text{fake_scores}] + \lambda * \text{GP}$ ($\lambda = \text{gp_weight} = 10$)

WGAN-GP utilise Adam(lr=0.0001, beta_1=0.5, beta_2=0.9)

2.6 Comparaison WGAN vs WGAN-GP

Critère	WGAN (Weight Clipping)	WGAN-GP (Gradient Penalty)
Contrainte Lipschitz	Clipping ± 0.01	Pénalité sur gradient
Stabilité	Bonne	Très bonne
Qualité images	Bonne	Meilleure
Learning rate	0.0002	0.0001
Beta_2	0.999 (défaut)	0.9
BatchNorm	Autorisée	Évitée dans le critic

TP2 — cGAN : Conditional GAN

3.1 Présentation et Motivation

Le **cGAN (Conditional GAN)** étend le GAN standard en conditionnant la génération par une information supplémentaire, typiquement un **label de classe y**. Cette approche permet de **contrôler le contenu de l'image générée** (ex : générer spécifiquement le chiffre 7). Le dataset utilisé est **MNIST** (chiffres manuscrits 0–9, 28×28 pixels, 60 000 images).

3.2 Dataset et Prétraitement

Paramètre	Valeur
Dataset	MNIST (chiffres 0–9)
Taille d'image	28 × 28 × 1 (niveaux de gris)
Taille train	60 000 images
Normalisation	[-1, 1] via $(x - 127.5) / 127.5$
Encodage labels	One-Hot Encoding (NUM_CLASSES=10)
Batch size	128

3.3 Architecture du Générateur Conditionnel

Le générateur conditionnel reçoit **deux entrées : le bruit z et le label one-hot y**. Ces deux vecteurs sont concaténés avant de traverser les couches de génération.

Couche	Configuration
Entrée bruit	Vecteur z de dimension LATENT_DIM=100
Entrée label	Vecteur one-hot de dimension 10
Concatenate	Fusion des deux entrées → vecteur de dim 110
Dense	7×7×128 units — BatchNorm — LeakyReLU(0.2) — Reshape(7,7,128)
Conv2DTranspose — 1	64 filtres, kernel 4×4, stride 2 → (14, 14, 64) — BatchNorm — LeakyReLU(0.2)
Conv2DTranspose — 2	1 filtre, kernel 4×4, stride 2, activation tanh → (28, 28, 1)

3.4 Architecture du Discriminateur Conditionnel

Le discriminateur conditionnel reçoit **l'image et le label associé**. Le label est transformé en une carte spatiale de même taille que l'image, puis concaténé comme canal supplémentaire.

Couche	Configuration
Entrée image	Tenseur (28, 28, 1)

Entrée label	Vecteur one-hot de dim 10
Embedding label	Dense(28×28×1) — Reshape(28, 28, 1)
Concatenate	Fusion image + label map → (28, 28, 2)
Conv2D — 1	64 filtres, stride 2 — LeakyReLU(0.2) + Dropout(0.3) → (14, 14, 64)
Conv2D — 2	128 filtres, stride 2 — LeakyReLU(0.2) + Dropout(0.3) → (7, 7, 128)
Dense sortie	Flatten + Dense(1) + sigmoid

3.5 Entraînement et Mécanisme Conditionnel

À chaque step d'entraînement, des labels aléatoires sont générés pour les fausses images, puis le discriminateur vérifie la cohérence **image↔label**. Cela force le générateur à produire des images correctement associées au label conditionnel.

Paramètre	Valeur
Epochs	30
Batch size	128
LATENT_DIM	100
NUM_CLASSES	10
Learning rate	0.0002 (Adam, beta_1=0.5)
Fonction de perte	BinaryCrossentropy

3.6 Génération Conditionnelle

Après entraînement, le générateur peut produire **n'importe quel chiffre sur commande**. Pour générer le chiffre d, on crée un vecteur one-hot [0,...,1,...,0] avec un 1 en position d, et on l'injecte dans le générateur.

Différence GAN vs cGAN :

- GAN : génère des images aléatoires sans contrôle du contenu.
- cGAN : conditionne la génération par un label y qui contrôle le contenu.

Pourquoi la conditionnement améliore-t-il la qualité ?

Le label réduit l'espace de recherche du générateur et guide l'apprentissage vers des distributions de classes précises.

Détecter le mode collapse :

- Toutes les images générées se ressemblent (peu de diversité).
- Perte du générateur chute très vite et reste stable.
- Perte du discriminateur tend vers zéro.

Solutions : dropout, minibatch discrimination, ajustement du LR.

Impact du learning rate :

- LR trop élevé → instabilité, oscillations adversariales.
- LR trop faible → convergence très lente.

TP3 — StyleGAN : Style-Based Generator

4.1 Présentation et Motivations

Le **StyleGAN** (Karras et al., NVIDIA, 2019) introduit une architecture radicalement différente : au lieu d'injecter le vecteur de bruit directement dans le générateur, il le transforme d'abord via un **réseau de mapping** pour obtenir un espace intermédiaire W beaucoup plus **disentangled**. Ce TP implémente une version simplifiée de StyleGAN1 sur **CIFAR-10** (10 classes, 50 000 images couleur 32×32 , redimensionnées en 64×64).

L'espace W vs l'espace Z

Espace Z (bruit direct) : très entangled — changer un peu z change tout dans l'image.

Espace W (après mapping) : beaucoup plus disentangled :

- Chaque dimension de w correspond plus clairement à une caractéristique visuelle.
- Permet un contrôle fin des attributs (pose, texture, couleur, style).
- Rend possible le style mixing (mélanger les styles de deux images).

4.2 Dataset CIFAR-10

Paramètre	Valeur
Dataset	CIFAR-10 (10 classes)
Taille originale	$32 \times 32 \times 3$ (RGB)
Taille après resize	$64 \times 64 \times 3$
Train / Test	50 000 / 10 000 images
Normalisation	$[-1, 1]$ via $\text{mean}=[0.5]*3$, $\text{std}=[0.5]*3$
Classes	avion, voiture, oiseau, chat, cerf, chien, grenouille, cheval, bateau, camion
Batch size	64

4.3 Hyperparamètres Globaux

Paramètre	Valeur
Z_DIM	128 (dimension du bruit latent)
W_DIM	128 (dimension de l'espace W)
IMAGE_SIZE	64
CHANNELS	3 (RGB)
EPOCHS	30
LR	$1e-4$ (Adam, betas=(0.0, 0.99))
NUM_SAMPLES	10 images générées à la fin

4.4 Mapping Network ($Z \rightarrow W$)

Le Mapping Network transforme le bruit z en vecteur de style w via **8 couches linéaires avec activation LeakyReLU(0.2)**. Cette architecture profonde permet de **déplier l'espace latent** (disentanglement) pour que chaque dimension de w corresponde à une caractéristique visuelle distincte.

Architecture Mapping Network

```
class MappingNetwork(nn.Module): #  $Z \rightarrow W$ 
    depth = 8 (couches linéaires)
    Chaque couche : Linear(in_dim  $\rightarrow$  w_dim) + LeakyReLU(0.2)
    Entrée :  $z$  de dimension Z_DIM=128
    Sortie :  $w$  de dimension W_DIM=128
```

→ Le réseau apprend une transformation non-linéaire complexe
→ Résulte en un espace W plus régulier et plus contrôlable que Z

4.5 AdaIN — Adaptive Instance Normalization

L'**AdaIN (Adaptive Instance Normalization)** est le mécanisme central d'injection de style. Il normalise les feature maps de l'image via Instance Normalization, puis les re-module selon le vecteur de style w .

Formule AdaIN

- Normalisation : $x_{\text{norm}} = \text{InstanceNorm}(x)$
→ normalise chaque canal indépendamment
 - Transformation du style w en deux vecteurs :
scale = Linear($w_{\text{dim}} \rightarrow \text{channels}$) — multiplicateur
bias = Linear($w_{\text{dim}} \rightarrow \text{channels}$) — décalage
 - Injection : $\text{output} = \text{scale} * x_{\text{norm}} + \text{bias}$
- Résultat : le style w contrôle directement l'apparence des features

4.6 Niveaux de Style

Résolution	Type de style contrôlé
4×4 à 8×8 (coarse)	Structure globale, pose, forme générale de l'objet
16×16 à 32×32 (middle)	Traits caractéristiques, composition, couleurs dominantes
64×64 et plus (fine)	Détails fins, textures, patterns locaux

4.7 Architecture du Générateur StyleGAN1

Le générateur commence avec un tenseur **constant appris** ($4 \times 4 \times 512$) et l'upscale progressivement via des blocs `StyleGAN1Block`. Chaque bloc effectue : Upsample $\times 2 \rightarrow$ Conv $3 \times 3 \rightarrow$ AdaIN $\rightarrow$ LeakyReLU $\rightarrow$ Conv $3 \times 3 \rightarrow$ AdaIN $\rightarrow$ LeakyReLU.

Bloc	Résolution entrée $\rightarrow$ sortie	Canaux
Const (appris)	$— \rightarrow 4 \times 4$	512
StyleGAN1Block 1	$4 \times 4 \rightarrow 8 \times 8$	$512 \rightarrow 256$
StyleGAN1Block 2	$8 \times 8 \rightarrow 16 \times 16$	$256 \rightarrow 128$
StyleGAN1Block 3	$16 \times 16 \rightarrow 32 \times 32$	$128 \rightarrow 64$
StyleGAN1Block 4	$32 \times 32 \rightarrow 64 \times 64$	$64 \rightarrow 32$
to_rgb (Conv 1×1)	64×64	$32 \rightarrow 3 + \tanh$

4.8 Discriminateur

Le discriminateur est un CNN standard (similaire DCGAN) qui réduit progressivement la résolution de 64×64 à un score scalaire :

Couche	Configuration
Conv2D — 1	$3 \rightarrow 64$ filtres, kernel 4, stride 2, padding 1 — LeakyReLU(0.2)
Conv2D — 2	$64 \rightarrow 128$ filtres — BatchNorm2d — LeakyReLU(0.2)
Conv2D — 3	$128 \rightarrow 256$ filtres — BatchNorm2d — LeakyReLU(0.2)
Conv2D — 4	$256 \rightarrow 512$ filtres — BatchNorm2d — LeakyReLU(0.2)
Conv2D — 5	$512 \rightarrow 1$ filtre, kernel 4, stride 1 $\rightarrow$ score scalaire

4.9 Entraînement

L'entraînement utilise `nn.BCEWithLogitsLoss()` et Adam avec des paramètres spéciaux : `betas=(0.0, 0.99)` (`beta_1=0` pour éviter les moments d'ordre 1 qui créent des oscillations, `beta_2=0.99` pour une moyenne mobile stable sur les gradients au carré).

Boucle d'entraînement

Pour chaque epoch :

1. Discriminateur :

- Générer $z \sim N(0,1)$, créer `fake = G(z).detach()`
- `loss_D = (BCE(D(real), ones) + BCE(D(fake), zeros)) / 2`
- `opt_D.zero_grad()  $\rightarrow$  loss_D.backward()  $\rightarrow$  opt_D.step()`

2. Générateur :

- Générer un nouveau z , créer `fake = G(z)`
- `loss_G = BCE(D(fake), ones)` [tromper D]
- `opt_G.zero_grad()  $\rightarrow$  loss_G.backward()  $\rightarrow$  opt_G.step()`

Aperçu des images générées toutes les 10 epochs (seed fixe)

TP4 — VAE : Variational Autoencoder

5.1 Présentation et Principe

Le **VAE (Variational Autoencoder)** est une approche radicalement différente des GANs : c'est un **modèle génératif probabiliste** qui apprend à encoder des données dans un espace latent structuré, puis à les décoder. Contrairement aux GANs, le VAE utilise un seul réseau entraîné par **maximisation de la vraisemblance** (ELBO — Evidence Lower BOUND). Ce TP utilise le dataset **MNIST** et explore deux dimensions latentes : 2D et 20D.

5.2 Hyperparamètres

Paramètre	Version 2D	Version 20D
LATENT_DIM	2	20
INPUT_DIM	784 (28×28)	784 (28×28)
HIDDEN_DIM	400	400
BATCH_SIZE	128	128
EPOCHS	30	30
LR	1e-3 (Adam)	1e-3 (Adam)
Encodeur	1 couche cachée	2 couches cachées
Décodeur	2 couches	3 couches

5.3 Architecture du VAE — Encodeur

L'encodeur transforme l'image aplatie (784D) en **deux vecteurs de dimension LATENT_DIM** : la **moyenne μ (mu)** et le **log de la variance $\log(\sigma^2)$ (logvar)**. Ces deux vecteurs paramétrisent une distribution gaussienne dans l'espace latent.

Encodeur (version 2D)

Input : $x \in \mathbb{R}^{784}$

- Linear(784 → 400) + ReLU
- fc_mu : Linear(400 → 2) — sortie : μ (moyenne)
- fc_logvar: Linear(400 → 2) — sortie : $\log(\sigma^2)$

Encodeur (version 20D) — plus profond :

- Linear(784 → 400) + ReLU
- Linear(400 → 200) + ReLU
- fc_mu : Linear(200 → 20)
- fc_logvar: Linear(200 → 20)

5.4 Reparameterization Trick

Le **reparameterization trick** est une astuce mathématique qui permet de **rétropropager les gradients à travers l'opération d'échantillonnage**, qui est normalement non-différentiable.

Formule et implémentation

Au lieu de sampler directement $z \sim N(\mu, \sigma^2)$ (non différentiable) :

```
std = exp(0.5 * logvar)    —  $\sigma = \exp(0.5 * \log(\sigma^2))$   
eps = N(0, 1)             — epsilon aléatoire (non appris)  
z = mu + std * eps        — différentiable par rapport à  $\mu$  et  $\sigma$ 
```

Avantage : les gradients passent par μ et σ , mais pas par ϵ .
→ Backpropagation possible même avec échantillonnage stochastique.

5.5 Architecture du Décodeur

Le décodeur reconstruit l'image à partir du vecteur latent z . L'activation **Sigmoid** en sortie garantit des pixels dans $[0,1]$, cohérents avec la normalisation des images MNIST.

Décodeur (version 2D)

```
z ∈ ℝ² → Linear(2 → 400) + ReLU  
        → Linear(400 → 784) + Sigmoid  
Sortie : x' ∈ [0,1]^784
```

Décodeur (version 20D) — plus profond :

```
z ∈ ℝ²⁰ → Linear(20 → 200) + ReLU  
        → Linear(200 → 400) + ReLU  
        → Linear(400 → 784) + Sigmoid
```

5.6 Fonction de Perte — ELBO

La perte du VAE combine deux termes complémentaires : la **Binary Cross-Entropy (BCE)** pour la reconstruction et la **KL Divergence** pour la régularisation de l'espace latent.

Décomposition de la perte VAE

Total Loss = Reconstruction Loss + KL Divergence

1. Reconstruction Loss (BCE) :

```
= BCE(x_recon, x, reduction='sum')  
→ Pénalise les pixels mal reconstruits pixel par pixel.  
→ BCE choisie car pixels MNIST normalisés dans  $[0,1]$ .
```

2. KL Divergence :

```
= -0.5 * Σ(1 + logvar - μ² - exp(logvar))
```

- Force la distribution $q(z|x) = N(\mu, \sigma^2)$ à rester proche de la distribution prior $N(0, 1)$.
- Garantit un espace latent continu et génératif.

Trade-off : BCE → bonne reconstruction ; KL → espace latent structuré

5.7 Visualisation de l'Espace Latent

5.7.1 Version 2D — Visualisation directe

Avec LATENT_DIM=2, l'espace latent peut être visualisé directement en 2D (scatter plot des vecteurs μ colorés par classe). On observe des **clusters distincts** pour chaque chiffre, organisés de manière continue.

5.7.2 Version 20D — Visualisation t-SNE

Avec LATENT_DIM=20, l'espace latent est de dimension 20. La visualisation nécessite une réduction de dimension par **t-SNE (t-distributed Stochastic Neighbor Embedding)** qui projette les vecteurs μ en 2D tout en préservant les structures de voisinage locales.

Paramètre t-SNE	Valeur
n_components	2
perplexity	30
n_iter	1000
random_state	42

5.8 Génération et Interpolation

Le VAE permet deux types de génération :

- **Génération directe** : Sampler $z \sim N(0,1)$ et le passer dans le décodeur. On génère 16 images avec z aléatoires dans l'espace latent.
- **Interpolation** : Calculer des points intermédiaires entre deux vecteurs z_1 et z_2 dans l'espace latent ($z_interp = (1-\alpha)z_1 + \alpha z_2$, $\alpha \in [0,1]$ en 10 étapes). Grâce à la structure continue de l'espace latent VAE, les images interpolées transitionnent de manière **progressive et cohérente** d'un chiffre à l'autre.

5.9 Comparaison 2D vs 20D

Critère	LATENT_DIM=2	LATENT_DIM=20
Visualisation latente	Directe (scatter 2D)	t-SNE (20D → 2D)
Qualité de reconstruction	Moins bonne (peu de capacité)	Meilleure (plus d'infos)
Qualité de génération	Images floues / simplifiées	Images plus précises
Séparation des clusters	Visible mais chevauchements	Très bien séparée (t-SNE)
Architecture encodeur	1 couche cachée	2 couches cachées

Architecture décodeur	2 couches	3 couches
Interpolation	Fluide (2D navigable)	Transition progressive

Synthèse Comparative — GAN vs VAE

6.1 Tableau Récapitulatif des Architectures

Modèle	Framework	Dataset	Espèce latent	Points clés
DCGAN	TensorFlow/Keras	LEGO 64×64	$z \in \mathbb{R}^{100}$	Conv2D/Transpose, BN, BCE
WGAN	TensorFlow/Keras	LEGO 64×64	$z \in \mathbb{R}^{100}$	Critic, Clipping, Wasserstein, Weight
WGAN-GP	TensorFlow/Keras	LEGO 64×64	$z \in \mathbb{R}^{100}$	Gradient Penalty, $\lambda=10$
cGAN	TensorFlow/Keras	MNIST 28×28	$z \in \mathbb{R}^{100}$	Conditionnement label y, One-Hot
StyleGAN	PyTorch	CIFAR-10 64×64	$z, w \in \mathbb{R}^{128}$	Mapping, AdaIN, Const appris
VAE 2D	PyTorch	MNIST 28×28	$z \in \mathbb{R}^2$	BCE+KL, Reparameterization
VAE 20D	PyTorch	MNIST 28×28	$z \in \mathbb{R}^{20}$	t-SNE, architecture plus profonde

6.2 GAN vs VAE — Comparaison Fondamentale

Critère	GAN	VAE
Approche	Jeu adversarial (générateur vs discriminateur)	Maximisation de la vraisemblance (ELBO)
Nombre de réseaux	2 (G + D)	1 (encodeur + décodeur)
Qualité d'image	Très haute (images nettes)	Plus floue (compromis reconstruction/régularisation)
Espace latent	Non structuré (non régulier)	Structuré (gaussien, continu, interpolable)
Génération contrôlée	Difficile sans cGAN	Naturelle (échantillonnage depuis $N(0,1)$)
Interpolation	Possible mais instable	Fluide et cohérente
Stabilité	Difficile (mode collapse, oscillations)	Entraînement stable
Perte	Adversariale (BCE ou Wasserstein)	Reconstruction (BCE) + KL Divergence

6.3 Problèmes Courants et Solutions

Problème	Modèle concerné	Solution
Mode collapse	Tous les GANs	Dropout, WGAN, minibatch discrimination

Gradient vanishing	DCGAN (discriminateur trop fort)	WGAN (distance Wasserstein)
Instabilité entraînement	Tous les GANs	WGAN-GP, LR faible, beta_1=0.5
Images floues	VAE	Augmenter LATENT_DIM, architecture plus profonde
Posterior collapse	VAE	Réduire le poids KL (β -VAE)
Capacity underuse	WGAN (weight clipping)	Remplacer par WGAN-GP
Espace latent non structuré	GAN standard	Utiliser un VAE ou un β -VAE

6.4 Choix du Modèle selon le Cas d'Usage

Guide de sélection

- Génération d'images de haute qualité sans contrôle de classe :
→ DCGAN (simple) ou WGAN-GP (stable et haute qualité)
- Génération contrôlée par classe (ex : générer spécifiquement le chiffre 7) :
→ cGAN (conditionnement par label one-hot)
- Contrôle fin du style visuel (texture, pose, couleur) :
→ StyleGAN (Mapping Network + AdaIN)
- Exploration de l'espace latent, interpolation, clustering :
→ VAE (espace latent continu et structuré)
- Génération + reconstruction + représentation compressée :
→ VAE (codec bidirectionnel encodeur-décodeur)

Conclusion

Ces quatre travaux pratiques ont permis d'explorer de manière progressive et détaillée les principales familles de **modèles génératifs pour la synthèse d'images** par intelligence artificielle.

Le TP1 a introduit le cadre fondamental des GANs avec le DCGAN, en montrant comment deux réseaux (générateur et discriminateur) s'entraînent mutuellement via un jeu adversarial. La variante WGAN a ensuite résolu les problèmes d'instabilité inhérents à la BCE en introduisant la distance de Wasserstein, et WGAN-GP en a amélioré encore la stabilité via la pénalité de gradient.

Le TP2 a étendu ce cadre avec le cGAN, permettant un contrôle précis du contenu généré par injection de labels de classe, ouvrant la voie à des applications pratiques de génération ciblée.

Le TP3 a introduit l'architecture StyleGAN, représentant l'état de l'art en génération d'images. Le réseau de mapping et l'AdaIN permettent un contrôle hiérarchique du style visuel, depuis la structure globale jusqu'aux détails fins.

Le TP4 a abordé une approche fondamentalement différente avec le VAE : un modèle génératif probabiliste qui apprend un espace latent continu et structuré. Les deux configurations (2D et 20D) ont permis de comprendre le compromis entre visualisabilité et capacité de représentation, ainsi que les propriétés d'interpolation et de génération propres aux VAE.